

Legallink API Contracts – Phase 1 (Citizen ↔ Backend)

Base URL

`http://localhost:3000/api` (development)

Staging/production base to be set via environment variable.

Authentication

- **Anonymous endpoints:** No `Authorization` header. Uses a **session cookie** (`legallink_session`) set automatically by the server.
- **Authenticated endpoints:** Require `Authorization: Bearer <jwt>` header. Token returned by the verify-OTP endpoint.

Session Cookie Convention

- Name: `legallink_session`
- Set by the server on **first anonymous request** (if no valid cookie present).
- Flags: `HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=86400` (24 hours)
- The cookie is automatically sent by the browser on subsequent requests; frontend does **not** need to manually attach it.
- On successful sign-up/sign-in, the session's anonymous Matters are claimed and a JWT token is returned; the session cookie may be dropped or kept for non-authenticated interactions (optional).

Response Envelope (All Endpoints)

All successful responses follow this wrapper (except WebSocket messages):

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-05-10T10:30:00Z",
    "requestId": "uuid"
  }
}
```

Error responses follow:

```
{
  "success": false,
  "error": {
    "code": "QUERY_REQUIRED",
    "message": "Human-readable message."
  }
}
```

1. Citizen Intake – Create Anonymous Matter

Endpoint: `POST /api/matter`

Auth: None (anonymous)

Description: Accepts a free-text legal query in Bengali or English, triggers AI classification + citation retrieval + bilingual response, and returns a matter with a preview of matching advocates.

Request

Headers:

- `Content-Type: application/json`
- (Optional) `Cookie: legallink_session=<existing-id>`

Body:

```
{
  "query": "যেহেতু আমি বীমার ডিজিটাল কপি দেখিয়েছিলাম, তাও কি পুলিশ আমাকে ১০,০০০ টাকা",
  "language": "bn"
}
```

Field	Type	Required	Description
<code>query</code>	string	Yes	Free-text legal situation, max 5000 characters.
<code>language</code>	string	Yes	<code>"bn"</code> or <code>"en"</code> . Explicit language choice.

Response

Status: 201 Created

Set-Cookie header (if new session).

Body:

```
{
  "success": true,
  "data": {
    "matterId": "mat_7h3k2j9x",
    "aiResponse": {
      "classification": {
        "matterType": "motor_vehicle / traffic_offence",
        "statute": "Motor Vehicles Act 1988",
        "userQuestion": "digital insurance proof validity",
        "involvesPolice": true,
        "incidentDate": null,
        "location": null
      },
      "citations": [
        {
          "source": "Motor Vehicles Act 1988",
          "section": "Section 139",
          "text": "Relevant excerpt...",
          "url": "https://indiacode.nic.in/..."
        }
      ],
      "responseEnglish": "Under the Motor Vehicles Act 1988...",
      "responseBengali": "মোটরযান আইন ১৯৮৮...",
      "disclaimer": "This is legal information, not legal advice. ..."
    },
    "matchingAdvocates": [
      {
        "advocateId": "adv_3k9m2x",
        "name": "Anirban S.",
        "enrolmentNumber": "WB/1234/2018",
        "practiceAreas": ["motor_vehicle", "traffic_offence"],
        "languages": ["bn", "en"],
        "districts": ["howrah"],
        "verificationStatus": "verified"
      }
    ],
    "nextSteps": {
      "canSave": "Sign up to save this matter and return to it later.",
      "canUpload": "Sign up to upload documents.",
    }
  }
}
```

```
    "canConnect": "Sign up to request a consultation with these advocat  
    "sessionExpiresAt": "2026-05-11T10:30:00Z"  
  }  
}  
}
```

Error cases:

- 400 - `QUERY_REQUIRED` (empty query)
 - 400 - `QUERY_TOO_LONG` (>5000 chars)
 - 400 - `LANGUAGE_UNSUPPORTED` (not `bn` / `en`)
 - 503 - `AI_UNAVAILABLE` (AI service down/timeout)
 - 200 (partial) – If AI can't classify, `aiResponse: null` and `message` field present.
-

2. Get Matter Details

Endpoint: `GET /api/matter/{matterId}`

Auth: Session cookie (anonymous) or JWT (after claim)

Description: Retrieve a previously created matter. Useful for the citizen to return to their result later (within the 24-hour session window, or after signing in).

Request

Path parameter: `matterId` (string)

Headers: `Cookie` (if anonymous) or `Authorization: Bearer <token>` (if claimed).

Response

Status: `200 OK`

Body: Same shape as the `data` field of the POST response, but the `aiResponse` and `matchingAdvocates` are the stored values (no re-computation). Adds a `status` field: `"session-owned"` or `"user-owned"` .

3. Authentication – Request OTP

Endpoint: `POST /api/auth/request-otp`

Auth: None

Description: Sends a one-time password to the citizen's phone for sign-up or sign-in.

Request

```
{
  "phone": "+919876543210"
}
```

Field	Type	Required	Description
phone	string	Yes	Phone number in E.164 format.

Response

Status: 202 Accepted

```
{
  "success": true,
  "data": {
    "expiresInSeconds": 300,
    "retryAfterSeconds": 30
  }
}
```

- The backend does **not** return any token here; it only triggers an SMS.
- Errors: 400 – INVALID_PHONE , 429 – TOO_MANY_REQUESTS

4. Authentication – Verify OTP & Claim Matters

Endpoint: POST /api/auth/verify-otp

Auth: Session cookie (to identify pending anonymous Matters)

Description: Verifies the OTP, creates/authenticates the user, and **claims** all session-owned Matters by linking them to the newly authenticated user.

Request

```
{
  "phone": "+919876543210",
```

```
"otp": "123456"
}
```

Headers:

- **Cookie:** `legallink_session=<session-id>` (mandatory to claim Matters)

Response

Status: `200 OK`

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs... ",
    "user": {
      "userId": "usr_9a2b3c",
      "phone": "+919876543210",
      "preferredLanguage": "bn",
      "createdAt": "2026-05-10T10:35:00Z"
    },
    "claimedMatters": ["mat_7h3k2j9x"]
  }
}
```

- `token` is a JWT to be sent as **Authorization: Bearer** for authenticated endpoints.
 - The frontend should store the token and include it in all subsequent requests that require authentication.
 - Errors: `400 - INVALID_OTP`, `401 - OTP_EXPIRED`.
-

5. Document Upload

Endpoint: `POST /api/matter/{matterId}/documents`

Auth: Bearer token (user must have signed up)

Description: Uploads a PDF or image (JPEG/PNG) associated with a matter.

Request

Headers:

- **Authorization:** Bearer <token>
- **Content-Type:** multipart/form-data

Form field:

- **file** – the file (max 10 MB)

Response

Status: 201 Created

```
{
  "success": true,
  "data": {
    "documentId": "doc_5f7h2k",
    "fileName": "challan.pdf",
    "fileType": "application/pdf",
    "size": 1024000,
    "uploadedAt": "2026-05-10T11:00:00Z"
  }
}
```

- Errors: 400 – FILE_TOO_LARGE , 400 – UNSUPPORTED_FILE_TYPE , 401 – unauthorized, 403 – matter does not belong to user.
-

6. Advocate Match List (Pagination)

Endpoint: GET /api/matter/{matterId}/advocates

Auth: Session cookie or JWT

Description: Retrieves the advocate shortlist for a matter. Used for pagination or refreshing the list after the initial creation.

Request

Query parameters (optional): **page** (default 1), **limit** (default 5, max 20)

Response

```
{
  "success": true,
  "data": {
    "advocates": [ ... ],
    "total": 2,
    "page": 1,
    "pages": 1
  }
}
```

Each advocate object is the same as in the initial POST response: `advocateId`, `name`, `enrolmentNumber`, `practiceAreas`, `languages`, `districts`, `verificationStatus`. **No contact details** (phone/email) are ever returned to an anonymous or unconnected citizen.

7. Consultation Request

Endpoint: `POST /api/consultations`

Auth: Bearer token

Description: Citizen requests a consultation with a specific advocate regarding a specific matter.

Request

```
{
  "matterId": "mat_7h3k2j9x",
  "advocateId": "adv_3k9m2x"
}
```

Response

Status: `201 Created`

```
{
  "success": true,
  "data": {
    "consultationId": "con_q1w2e3",
    "status": "requested",
    "matterId": "mat_7h3k2j9x",
    "advocateId": "adv_3k9m2x",
  }
}
```



```
    "createdAt": "2026-05-10T12:00:00Z"
  }
}
```

- The backend must verify that the matter belongs to the user and that the advocate is verified.
 - Errors: 401 , 403 (matter ownership), 404 (advocate not found), 409 (consultation already exists for this matter).
-

8. Consultation Status / Accept-Reject (Shared – Person 1 implements)

Endpoint: PUT /api/consultations/{consultationId} (Advocate side)

Auth: Advocate Bearer token

Request body:

```
{
  "action": "accept" | "decline",
  "declineReason": "optional string if declined"
}
```

Response: Updated consultation object with status: "accepted" or "declined" .

- This endpoint is built by Person 1 (Advocate backend), but Person 2 and 3 need to know its existence so the citizen frontend can later poll or be notified (via WebSocket) when status changes.
-

9. WebSocket Chat (Matter-Scoped)

Connection: ws://localhost:3000/ws/consultation/{consultationId}

Auth: Query parameter token (JWT) passed during handshake.

Sample: ws://localhost:3000/ws/consultation/con_q1w2e3?token=eyJ...

Both citizen and advocate connect to the same endpoint. The server validates the token, verifies the user is a participant of the consultation, then upgrades the connection.

Message Format (Client → Server)

```
{
  "type": "message",
  "payload": {
    "text": "Hello, Advocate. I wanted to ask about..."
  }
}
```

Optional: `"type": "typing"` with `payload: { "isTyping": true }`.

Message Format (Server → Client)

```
{
  "type": "message",
  "payload": {
    "messageId": "msg_1a2b3c",
    "sender": "citizen" | "advocate",
    "text": "Hello, Advocate...",
    "timestamp": "2026-05-10T12:05:00Z",
    "moderationStatus": "approved"
  }
}
```

If moderation flags the advocate's message, the message is **not** broadcast to the citizen until reviewed. The advocate may receive a placeholder: `"moderationStatus": "pending"`.

10. Advocate Onboarding (Person 1's Domain – Reference Only)

For completeness, Person 1 will implement advocate-facing endpoints. The citizen frontend does **not** call these, but they must exist for Phase 1's jury submission. Person 1's contract will be:

- `POST /api/advocate/register` (initiate sign-up)
- `PUT /api/advocate/profile` (update the 6-field schema + upload CoP)
- `POST /api/advocate/verify` (submit for admin verification)
- `GET /api/advocate/dashboard` (profile summary, stats)
- `GET /api/advocate/consultations` (incoming requests)
- `PUT /api/advocate/consultations/{id}` (accept/reject)
- `GET /api/advocate/messages?consultationId=...` (chat history)

Person 2 and 3 do **not** need the internal details of these for parallel work, but they should know the advocate's acceptance action changes a consultation's status to **"accepted"** , which then enables WebSocket chat.

Summary of Contract Deliverables

#	Method	Endpoint	Auth	Built by	Used by
1	POST	/api/matter	Cookie	Person 2	Person 3
2	GET	/api/matter/{id}	Cookie/JWT	Person 2	Person 3
3	POST	/api/auth/request-otp	None	Person 2	Person 3
4	POST	/api/auth/verify-otp	Cookie	Person 2	Person 3
5	POST	/api/matter/{id}/documents	JWT	Person 2	Person 3
6	GET	/api/matter/{id}/advocates	Cookie/JWT	Person 2	Person 3
7	POST	/api/consultations	JWT	Person 2	Person 3
8	PUT	/api/consultations/{id}	Advocate JWT	Person 1	(internal)
9	WS	/ws/consultation/{id}	JWT (query)	Shared module?	Person 3 (citizen), Person 1 (advocate)